Assignment 1: Policy Gradient, Parts 1 and 2

GitHub: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 Implementation

Both tasks use PyTorch Monte Carlo policy gradients and complete episodes. CartPole uses `CartPole-v1`; Pong uses `ALE/Pong-v5`, the namespaced Pong-v5 environment. Actions are sampled during training and evaluation.

For decision t, s_t is the observed input, a_t the action, r_t its subsequent reward, and T the episode length. The policy $\pi_\theta(a \mid s)$ is a neural network with parameters θ. With discount γ, the objective and reward-to-go are

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

For a batch of B episodes, Adam minimizes

$$\mathcal{L} = -\frac{c}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})(G_t^{(i)} - b_t^{(i)}).$$

Here i indexes episodes, b_t is the baseline (zero in Part 1), and c is a fixed scale. Returns and baselines are detached from actor gradients; no return centering or standardization is used. Plots show *undiscounted* episode rewards, as requested.

Setting	CartPole	Pong
Input	4 physical observations	6,400 frame-difference pixels
Hidden layer; output	64 tanh; 2 logits	200 ReLU; 2 logits
Discount γ; scale c	0.95; 0.05	0.99; 1
Actor Adam learning rate	0.01	0.001
Episodes per update; total	32; 8,000	8; 20,000
Gradient-norm cap	10	5
Training seeds	1668, 1669, 1670	1668
Hardware	CPU	One GPU

Pong uses only actions 2 and 3 (RIGHT and LEFT), with no startup FIRE/NOOP. Frames are cropped and downsampled to 80×80, backgrounds 144/109 removed, and foreground binarized using the assignment's preprocessing. Consecutive binary frames are subtracted to supply motion; the reset input is zero. Default frame skip 4 and sticky-action probability 0.25 are retained. Full games are episodes: returns never reset at points. Gradients accumulate over all eight games before one update.

Pong's alternative rate 0.001 gives a smaller update: on identical game data, mean policy KL divergence after one update was 1.996×10^{-6}, versus 3.423×10^{-4} at 0.01. This motivates a conservative rate but does not establish better long-run stability. Clipping affected 61.68% of basic Pong updates and 18.64% with a baseline; CartPole never reached its cap.

Evaluation. Each main result uses the final fixed-budget checkpoint and 500 fresh episodes, without learning. Test reset seeds are separate from training and shared across methods. CartPole seed 1668 was selected before training for the main plots; all three seeds are reported below. SD denotes population SD across the 500 episode rewards.

2 Part 1(a): CartPole-v1

The final primary policy scored **500.000 ± 0.000**. All 500 evaluations reached the 500-step limit. The untrained neural policy scored 20.926 ± 11.467 and a uniform random policy 22.782 ± 12.058 over 500 episodes each. Training improved performance substantially, but included large temporary declines.

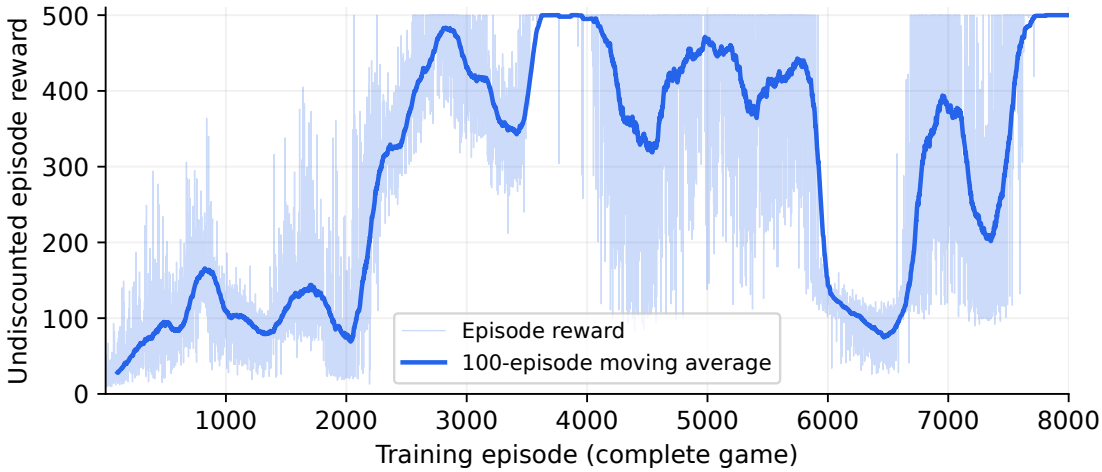


Figure 1: CartPole without a baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 episodes.

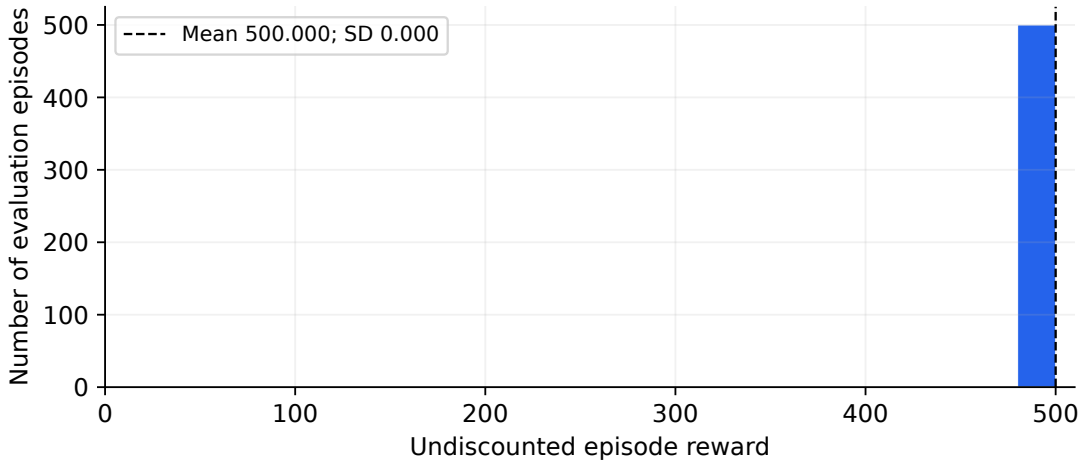


Figure 2: CartPole without a baseline: 500 final-policy evaluation rewards. Mean 500.000; population standard deviation 0.000.

3 Part 1(b): Pong-v5

The final policy scored **-18.610 ± 1.681**, ranging from −21 to −10. All 500 games were losses; none was truncated. An untrained reference scored −20.250 ± 0.935 over 32 games. Performance improved modestly, but **the policy did not learn to win**.

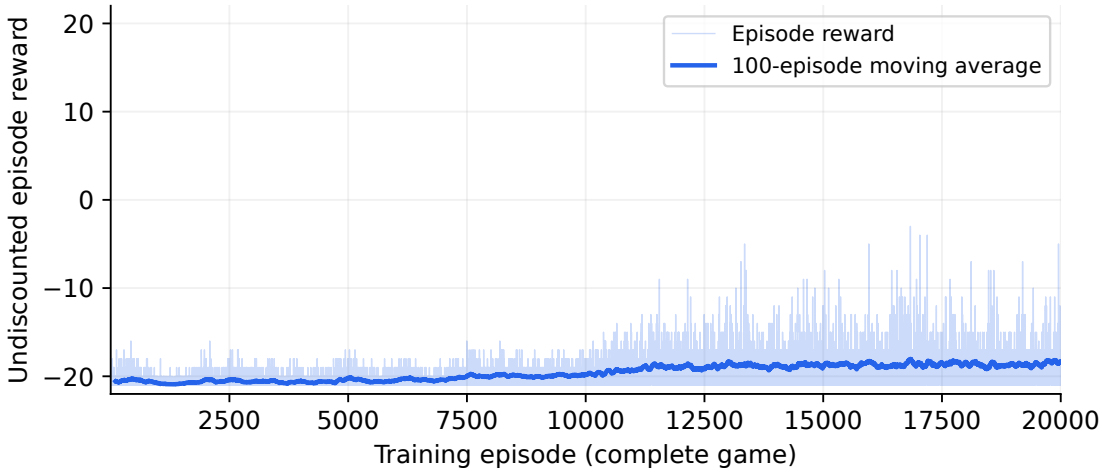


Figure 3: Pong without a baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.

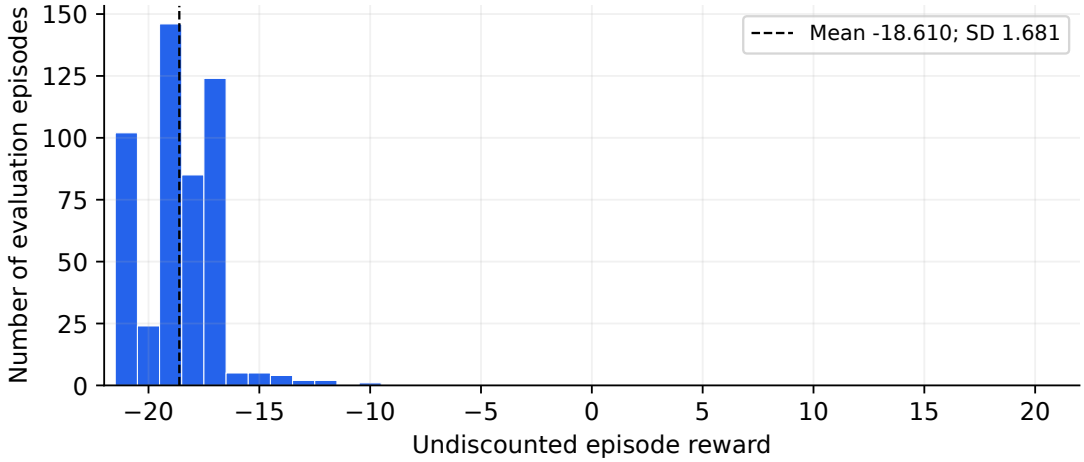


Figure 4: Pong without a baseline: 500 final-policy evaluation rewards. Mean -18.610; population standard deviation 1.681.

4 Part 2(a): CartPole with a baseline

The baseline is a separate network with inputs $(s_t, t/500)$, 64 tanh units and one output predicting G_t. Predictions use weights from earlier batches and are frozen for the actor update. Afterward, five Adam steps at rate 0.01 fit Monte Carlo returns by mean squared error. Actor settings and initial parameters match Part 1.

The final model scored **260.308 ± 57.013**. Four episodes reached 500; 496 ended in physical failure. This primary baseline run performed worse than the basic method.

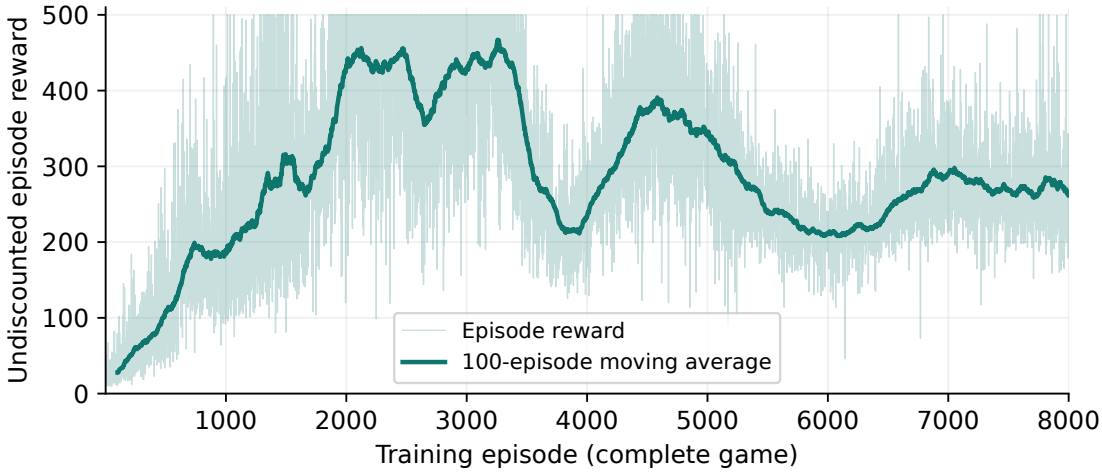


Figure 5: CartPole with a learned baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 8,000 episodes.

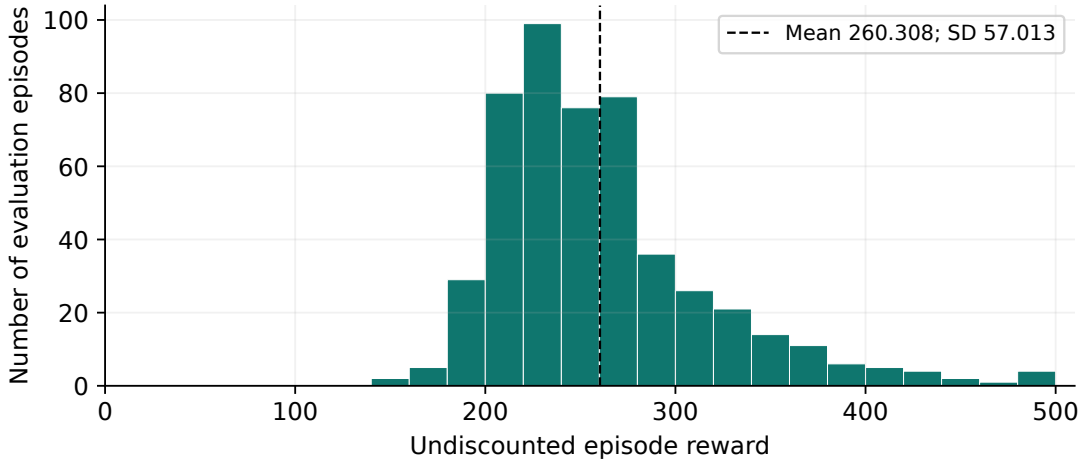


Figure 6: CartPole with a learned baseline: 500 final-policy evaluation rewards. Mean 260.308; population standard deviation 57.013.

5 Part 2(b): Pong with a baseline

The baseline is a lagged scalar exponential moving average. Update k uses b_k, starting at $b_0 = 0$; afterward,

$$\bar{G}_k = \frac{\sum_{i,t} \gamma^t G_t^{(i)}}{\sum_{i,t} \gamma^t}, \qquad b_{k+1} = 0.95 b_k + 0.05 \bar{G}_k.$$

Thus the baseline uses only previous batches when forming an actor gradient. An action-independent baseline preserves the expected unclipped gradient because $\mathbb{E}_a[b \nabla_\theta \log \pi_\theta(a \mid s)] = b \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$. Here ∇_θ denotes differentiation with respect to policy parameters. Clipping can alter this expectation.

The final score was **-18.446 ± 1.851**, only 0.164 above Part 1. All 500 games were losses (range −21 to −5), with no truncations. One training seed cannot establish a general benefit.

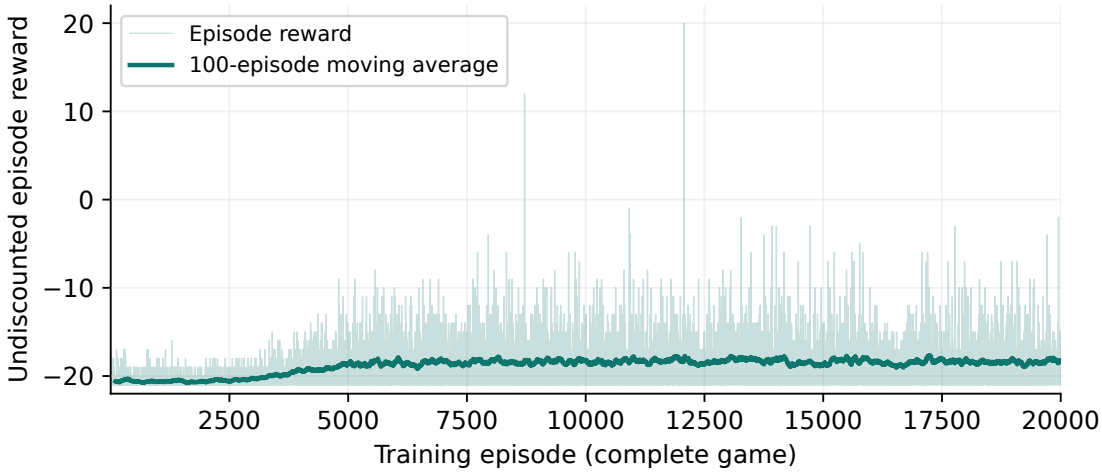


Figure 7: Pong with an EMA baseline: every training episode reward and trailing 100-episode mean. Seed 1668; 20,000 full games.



Figure 8: Pong with an EMA baseline: 500 final-policy evaluation rewards. Mean -18.446; population standard deviation 1.851.

6 Additional experiments and interpretation

Training seeds and learning rate

Method	Seed	Mean	SD	Fraction at 500
No baseline	1668	500.000	0.000	1.000
No baseline	1669	271.824	28.856	0.000
No baseline	1670	483.886	65.298	0.928
Learned baseline	1668	260.308	57.013	0.008
Learned baseline	1669	360.516	79.141	0.100
Learned baseline	1670	349.940	76.321	0.092

Across the three seeds, mean evaluation reward was 418.570 without a baseline and 323.588 with it. The baseline improved seed 1669 but not the others.

Further runs changed only the actor rate, retaining the same three seeds, budgets and 500-episode evaluations; the critic rate stayed 0.01:

Actor rate	No baseline: mean across seeds	Learned baseline: mean across seeds
0.01	418.570	323.588
0.005	163.178	357.661
0.002	247.102	264.481

Smaller rates did not consistently improve final performance. These are exploratory comparisons across three seeds, not evidence of an optimal rate.

Were CartPole's dips just episode noise?

Both original seed-1668 runs were replayed exactly: all 8,000 rewards and final parameter hashes matched. Frozen checkpoints before and during selected dips were then evaluated on 500 fresh paired reset/action seeds. Basic-policy means fell from 453.108 (episode 5,120) to 83.114 (6,400): paired change -369.994, bootstrap 95% interval $[-377.430, -362.038]$. Baseline-policy means fell from 440.466 (3,072) to 215.686 (6,144): change -224.780, interval $[-231.886, -217.436]$.

Holding the earlier basic policy fixed for 2,000 fresh episodes gave 100-episode averages of 430.11–472.29; continued training reached 74.64. The baseline's frozen range was 413.00–453.82, versus a continued-training minimum of 211.37. Policy quality changed beyond ordinary fixed-policy episode noise. Noisy updates may cause such changes, but this experiment does not identify the optimization mechanism. Checkpoints were selected after inspecting training curves; intervals are conditional on these policies, not uncertainty across training seeds.

What the discounted objective rewards

CartPole's discounted return is $(1 - 0.95^T)/0.05$, approaching 20 even for episodes shorter than 500. The primary baseline model averaged 19.999743 despite its lower undiscounted score. In Pong, discounted evaluation return improved from -1.309180 at initialization to 0.313913 (basic) and 0.293346 (baseline), while full-game scores remained poor. At $\gamma = 0.99$, rewards 1,000 decisions away receive weight 4.32×10^{-5}. This suggests a credit-assignment limitation, but is not a causal test.

7 Edge cases and quantitative validation

CartPole: both methods

Returns and gradient. Rewards $(1, 1, 1)$ at $\gamma = 0.95$ must give returns $(2.8525, 1.95, 1)$; the code matches. Setting $\gamma = 0$ gives immediate rewards. In an enumerable two-step task with binary-action reward and action probability $p = \text{sigmoid}(\theta)$, the analytic gradient is $(1 + \gamma)p(1 - p)$. Enumeration gave 0.471188756 versus analytic 0.471188748, with and without a constant baseline. A rewarded-action update increased its probability from 0.500000 to 0.502500, checking the loss sign.

End conditions. Forced cart-position and pole-angle violations terminated the episode and retained the terminal +1 reward. A one-step time limit truncated without physical failure. Either flag ends return accumulation, with zero continuation. Fixed reset seeds and ten fixed actions reproduced identical states. The saved ten-decision trace records states, action probabilities, actions and rewards for inspection.

Baseline mechanism. At a frozen initial actor, a baseline fitted on 512 separate episodes was evaluated on 512 paired held-out trajectories. Summed parameter-gradient variances fell from 5.338262 to 0.842421 (84.22%). The empirical mean-gradient difference was 0.0474, versus paired standard-error magnitude 0.1176. This supports variance reduction at that actor; it does not validate sustained reduction by the critic's five-step online fitting schedule. The controlled replay experiment above separately tests whether training dips reflect real policy changes.

Pong: both methods

Episode boundaries and actions. Rewards $(0, 1, 0, -1)$ at $\gamma = 0.99$ must yield

$$(G_0, G_1, G_2, G_3) = (0.019701, 0.0199, -0.99, -1).$$

The implementation matched within 10^{-6}. This checks that returns cross scoring boundaries. Zero discount recovered immediate rewards; the two-step analytic-gradient test agreed within 10^{-7}. Zero returns produced zero basic-policy gradient. Complete games reached scoring events and termination using only actions 2/3; workers reject other actions.

Images and replay. Known synthetic pixels verified crop, downsampling, background removal and foreground preservation without modifying the source image. Real inputs were zero at reset and otherwise in $\{-1, 0, 1\}$. Resetting frame history prevents leakage across games. Identical emulator seeds and fixed actions reproduced frame/reward hashes with sticky actions retained. Saved traces include the first ten decisions, a scoring event and the ending, with corresponding image differences.

Baseline mechanism. A separately fitted initial constant reduced summed gradient variance from 369.320 to 20.763 on 16 held-out games (94.38%). This candidate is distinct from the training EMA, which starts at zero. At 4,096, 8,192, 12,288 and 16,384 games, the actual EMA's paired variance ratios (with/without baseline, at the same frozen actor) were 0.873, 1.017, 0.924 and 1.011. It did not consistently reduce variance. These small 16-game probes have no variance-ratio confidence intervals and measure unclipped gradients.

Recovery and resources. Restoring actor and Adam state and repeating the next fixed-data update reproduced parameter hashes exactly. Preflight passed all 15 checks and verified one visible GPU. Both methods used that GPU sequentially with eight CPU emulator workers; training and evaluation completed in 2.64 hours. These checks support implementation correctness, while the game scores establish that learning remained inadequate.

8 Run instructions

Paths below are relative to the repository's `assignment1/` directory. Use a fresh copy for training to preserve saved results. Python 3.12, PyTorch 2.14.0, NumPy 2.5.3 and Gymnasium 1.2.2 were used; Pong also used ALE 0.11.2.

CartPole: install requirements, train all six models, evaluate each for 500 episodes, then run the numerical and environment tests:

```
cd mlds_assignment1_cartpole
python -m pip install -r requirements.txt
python cartpole.py suite --episodes 8000 --lr 0.01
python validate.py
jupyter lab cartpole_plots.ipynb
```

Validation evidence is in `validation/validation.json`; the notebook plots saved results without retraining. The controlled dip experiment is reproduced with `python instability_experiment.py`, followed by `python plot_instability_experiment.py`; see its saved `plan.json` and README for seeds and bootstrap details. The separate `cartpole_lr005` and `cartpole_lr002` folders each contain `run.sh`, which runs the six lower-rate experiments, validation and plotting.

Pong: in a fresh directory containing its source, config and requirements, activate a Python environment and install requirements. Select one GPU; set its UUID in `config.json` and `supervise.sh`. Then:

```
export CUDA_VISIBLE_DEVICES=<single-GPU-UUID>
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python run_pong.py preflight
nohup setsid bash supervise.sh > nohup.log 2>&1 < /dev/null &
```

Preflight executes the Pong checks above. Existing checkpoints resume rather than restart. See `preflight/results.json`, `runs/*/training.csv`, `runs/*/evaluation/test_final.json`, and `diagnostics/` for quantitative evidence. The source README documents recovery and traces.

Report: run `python build_assets.py`, then compile `POLICY_GRADIENT_REPORT.tex` with Tectonic or pdfLaTeX. Run `python combine_submission.py` to join the shuttle and policy-gradient PDFs.

9 Experiments with three additional months

CartPole: use more independent training seeds and separate tuning/test resets. Compare learning rates, batch sizes and baseline fitting schedules under matched budgets; measure critic error and paired gradient variance throughout training. Stress-test initial angles, velocities and altered physical parameters to assess generalization beyond standard resets.

Pong: keep v5, actions 2/3 and $\gamma = 0.99$ while comparing frame differences against frame stacks and a small convolutional policy. Measure gradient contributions by episode time to test delayed-credit limitations. Tune learning rate, clipping and batch size on validation seeds, then repeat training across seeds with untouched 500-game tests.

Baselines: compare scalar, state-dependent and time-dependent predictions at identical frozen actors, fitting on independent data. Use hundreds of paired trajectories and bootstrap variance-ratio intervals. Compare clipped and unclipped estimators to determine whether variance reduction translates into better control.